

TKOM Projekt Wstępny - PyScript

Andrzej Pultyn

Semestr 25L

Spis treści

1	Zadanie	2
2	Wymagania projektu	2
2.1	Wymagania funkcjonalne	2
2.2	Wymagania нефункционалне	3
3	Opis realizowanych funkcjonalności	3
3.1	Charakterystyka języka	3
3.2	Typy danych	3
3.3	Zmienne	5
3.4	Wyrażenia arytmetyczne i logiczne	5
3.5	Instrukcje warunkowe	6
3.6	Pętle	6
3.7	Własne funkcje	7
3.8	Funkcje wbudowane	7
3.9	Obiektowość	8
3.10	Komentarze	8
3.11	Słownik	9
3.11.1	Charakterystyka	9
3.11.2	Tworzenie nowego słownika	9
3.11.3	Metody	10
3.12	Zapytania LINQ	10
4	Obsługa błędów	11
5	Sposób uruchomienia	12
6	Składnia realizowanego języka	12
6.1	Słowa kluczowe języka	12
6.2	Gramatyka	13
7	Testowanie	13

1 Zadanie

Zadaniem jest napisanie interpretera do języka z wbudowanym typem słownika z określoną kolejnością elementów. Kolejność elementów w słowniku jest określana w momencie jego tworzenia - argumentem konstruktora słownika jest funkcja określająca, w jakiej kolejności mają być dodawane elementy. Możliwe powinny być podstawowe operacje na słowniku (dodawanie, usuwanie, wyszukiwanie elementów według klucza, sprawdzenie, czy dany klucz znajduje się w słowniku itd.), iterowanie po elementach oraz wykonywanie na słowniku zapytań w stylu LINQ.

2 Wymagania projektu

2.1 Wymagania funkcjonalne

Język powinien zawierać podstawowe elementy języka programowania oraz umożliwiać wykonanie podstawowych operacji programistycznych:

- typy całkowitoliczbowe oraz zmiennoprzecinkowe
- stałe znakowe
- wyrażenia arytmetyczne i logiczne
- instrukcje warunkowe
- pętle
- funkcje (wraz z rekursywnym wywołaniem)
- namiastka obiektowości (dodatkowe typy danych z konstruktorami, polami/metodami, dostępem do składowych przez kropkę)
- komentarze
- zmienne

Oprócz podstawowych elementów, język powinien posiadać klasę słownika o następujących funkcjonalnościach:

- przechowywanie par klucz-wartość
- dodawanie elementów
- usuwanie elementów
- wyszukiwanie elementów według klucza
- sprawdzanie, czy dany klucz znajduje się w słowniku
- iterowanie po elementach słownika
- wykonywanie zapytań w stylu LINQ

Interpreter powinien zapewniać:

- obsługę błędów języka z wartościowymi informacjami zwrotnymi
- czytanie kodu z plików

2.2 Wymagania niefunkcjonalne

- wygoda pisania i czytania kodu
- brak kończenia działania interpretera na skutek niespodziewanego błędu

3 Opis realizowanych funkcjonalności

3.1 Charakterystyka języka

Język będzie dynamicznie oraz silnie typowany. Zakres widoczności i bloki kodu opierać się będzie na nawiasach klamrowych, tak jak w języku C czy Java. Białe znaki, nie licząc stałych tekstowych i komentarzy, nie mają znaczenia. Każde wyrażenie kończyć się będzie średnikiem.

3.2 Typy danych

Język zawierać będzie następujące typy danych:

- **Int** - *integer* - typ liczb stałoprzecinkowych:

```
{
    10;
    -50 + 29;
}
```

Język zawierać będzie funkcję wbudowaną **toInt(wartość)**, która umożliwi rzutowanie wartości *String* lub *Float* na typ *Int*.

- **Float** - typ liczb zmiennoprzecinkowych. Wartości zmiennoprzecinkowe **muszą** zawierać kropkę, oraz przynajmniej jedną cyfrę po przecinku:

```
{
    1.0;
    -15.38;
}
```

Analogicznie do typu *Int*, język zawierać będzie funkcję **toFloat(wartość)**, umożliwiającą rzutowanie z typów *Int* oraz *String* na *Float*.

- **String** - typ stałych znakowych. Stałe umieszczone są w cudzysłowie:

```
{
    "Hello World!";
    "Typing with quotes: \"\" and backslash: \\"";
}
```

Ten typ również zawierać będzie funkcję rzutującą: **toString(wartość)**. Przyjmować ona będzie wartości o **dowolnym** typie w języku. Zawierać będzie też funkcję *length()*, zwracający długość stałej.

- **Bool** - *boolean* - typ boolowski:

```
{
    True;
    False;
    5 < 4;    // False
}
```

- **List** - typ nieuporządkowanej listy. Przyjmować może ona dowolne typy wartości z języka, również inne listy, tworząc listę zagnieżdżoną:

```
{
    [];
    [1, "Hello", -10.5, ("key": 5),
     ["hello", "from", "sublist"],
     {("name": "dict"), ("value": 5)}];
}
```

Na typie listy można będzie wykonać następujące operacje:

- *get(indeks)* - pobranie elementu na podanym indeksie
- *set(indeks, wartość)* - podmiana wartości na podanym indeksie na inną
- *add(wartość)* - dodanie elementu na koniec listy
- *remove(indeks)* - usunięcie elementu o podanym indeksie z listy
- *length()* - zwracający długość listy

- **Dict** - *dictionary* - typ słownika. Zawierać może wyłącznie obiekty typu *Item*:

```
{
    {};
```

```
    {
        ("first_key": 5),
        ("another_key": "value"),
        ("one_more": [1, 5, "hello"]),
        ("last_one": {("key": "value")});
    };
}
```

Funkcjonalności słownika opisane w dedykowanym podrozdziale.

- **Item** - typ obiektu przechowywanego w słowniku. Zawiera parę klucz oraz wartość (dowolnych

typów):

```
{
    ("key": "value");
    (1: 10);
}
```

Typ zawierać będzie 2 funkcje wbudowane: *key()*, zwracającą klucz elementu, oraz *value()*, zwracający jego wartość.

- **Funkcja** - funkcja również jest typem danych, który może być przekazywany do innych funkcji.

3.3 Zmienne

Zmienne definiowane są podaniem identyfikatora, a następnie przypisaniem do niej wartości:

```
{
    a = "Hello there";
    my_list = [1, "Hello", a];
}
```

Zmienne widoczne są w zdefiniowanym bloku oraz wewnątrz. Zmienne przechowują **referencje** do obiektów. Aby wykonać kopię obiektu, należy użyć na nim wbudowanej funkcji *copy()*.

3.4 Wyrażenia arytmetyczne i logiczne

Wyrażenia mogą być stałymi odpowiednich typów, wartościami zwracanymi przez funkcje, lub ich kombinacjami z operatorami:

Operator	Znaczenie	Priorytet	Asocjacja
()	Zawołanie obiektu	9	Lewostronna
.	Składowa obiektu	8	Lewostronna
!	Negacja logiczna	7	Prawostronna
-	Negacja arytmetyczna	7	Prawostronna
*	Mnożenie arytmetyczne	6	Lewostronna
/	Dzielenie arytmetyczne	6	Lewostronna
+	Dodawanie arytmetyczne	5	Lewostronna
-	Odejmowanie arytmetyczne	5	Lewostronna
>	Większy niż	4	Lewostronna
>=	Większy lub równy	4	Lewostronna
<	Mniejszy niż	4	Lewostronna
<=	Mniejszy lub równy	4	Lewostronna
==	Równy	4	Lewostronna
!=	Nierówny	4	Lewostronna
and	Logiczny AND	3	Lewostronna
or	Logiczny OR	2	Lewostronna
=	Operator przypisania	1	Prawostronna
+=	Operator przypisania	1	Prawostronna
-=	Operator przypisania	1	Prawostronna

Kilka ważnych uwag dotyczących operatorów:

- operatory działań arytmetycznych (+, -, *, /) wymagają identycznych typów liczbowych po obu stronach (*Int* lub *Float*)
- operator dodawania działa również na typach *String*, *List* oraz *Dict* jako konkatenacja (w przypadku pokrycia się kluczy przy konkatenacji dwóch słowników zostanie zgłoszony błąd)
- operatory porównania (==, !=, <, <=, >, >=) działają również na typie string. Równość i nierówność sprawdzają, czy stringi są identyczne, a większy/mniejszy rozstrzygany jest poprzez porządek alfabetyczny
- operatory równości i nierówności (==, !=) działają na różnych typach danych, gdzie różne typy danych traktowane są jako nierówne obiekty

3.5 Instrukcje warunkowe

Przy tworzeniu instrukcji warunkowych, można skorzystać z 3 poleceń, znanych z innych języków programowania: *if*, *else if*, *else*:

```
{
    if (a < 4) {
        do_something();
    } else if (a > 4) {
        do_something_else();
    } else {
        do_something_differently();
    }
}
```

3.6 Pętle

Pętle skonstruować będzie można na 2 sposoby:

- słowem *while*, która będzie wykonywana dopóki podany warunek jest prawdziwy:

```
{
    a = 0;
    while (a < 10) {
        do_something_ten_times();
        a += 1;
    }
}
```

- słowem *for*, która służy do iteracji po elementach listy lub słownika:

```
{
    my_dict = {"first": 1}, {"second": 10};
    for element in my_dict {
        print(element.key()); // "first" "second"
    }
}
```

```
}
```

Iterujemy po elementach po kolei. W przypadku słownika, elementy są sortowane podczas dodawania elementów, więc iteracja po kolei da odpowiednią kolejność.

3.7 Własne funkcje

Funkcje definiujemy, przypisując ją do zmiennej:

```
{
    my_func = function(arg1, arg2) {
        return arg1 + arg2;
    }

    print(my_func(5, 10)) // 15;
}
```

Funkcje mogą być wywoływane rekursywnie:

```
{
    factorial = function(n) {
        if (n == 1) {
            return 1;
        }
        return n * factorial(n-1);
    }
}
```

Oraz mogą być przekazywane do innych funkcji jako argumenty:

```
{
    passed_func = function(a, b) {
        return a + b;
    }

    another_function = function(func, a) {
        return func(a, 5);
    }

    print(another_function(passed_func, 10));
    // 15
}
```

Argumenty do funkcji przekazywane będą **przez referencję**.

3.8 Funkcje wbudowane

Język będzie zawierać następujące funkcje wbudowane:

- *print(wartość1, wartość2, ...)* - wypisanie na standardowe wyjście podanych wartości (maksymalnie 10). Elementy złożone takie jak *List* czy *Dict* również mogą być wypisywane. W przypadku

zagnieżdżonych elementów złożonych, wypisywany będzie ich typ wraz z jego długością:

```
{
    my_list = [1, 5, [1, "Hello", {"key", 5}], {"key", 2}, ("diff", 3)]

    print(my_list);
    // [1, 5, List(3), Dict(1)]

    print(my_list.get(3);
    // {"key", 2}, ("diff", Dict(0))}

    print(my_list.get(2);
    // [1, "Hello", Dict(1)]
}
```

- *toSomething(val)* - opisane w sekcji typy danych, funkcje składowe obiektów, pozwalające na rzutowanie jednych typów na inne:
 - *toString(val)* - rzutująca **dowolny** typ danej na wartość *String*
 - *toInt(val)* - rzutująca wartość **String** lub **Float** na wartość typu *Int*. W przypadku rzutowania z typu *Float*, część po przecinku zostanie po prostu ucięta (np. z 10.5 będzie 10, a z -9.55 będzie -9)
 - *toFloat(val)* - rzutująca wartość **Int** lub **String** na *Float*
- *defaultSort(item1, item2)* - domyślna funkcja sortowania słownika, opisana w sekcji o słowniku
- *typeOf(variable)* - funkcja zwracająca *stringa* z nazwą typu obiektu
- *<zmienna>.copy()* - utworzenie kopii obiektu pod daną zmienną

3.9 Obiektość

Język zawiera elementy obiektości, mamy specjalne typy (*Item*, *Dict*, *List*), które zawierają funkcje składowe (np. *List.get(idx)*). Mamy również konstruktor do klasy *Dict*.

3.10 Komentarze

Komentarze w języku można pisać na 2 sposoby, identyczne jak w języku C:

- podwójny slash - tworzący komentarz do końca linii
- */* */* - komentarz inline od jednego znaku do drugiego

Będą one ignorowane jako *whitespace*.

3.11 Słownik

3.11.1 Charakterystyka

Słownik jest kolekcją, przechowującą elementy typu *Item* o unikalnych kluczach. Przy tworzeniu słownika można podać mu funkcję sortującą, decydującą o kolejności elementów. Powinna ona być standardową funkcją porównawczą, czyli przyjmować 2 elementy typu *Item*, oraz zwracać:

- -1, jeżeli pierwszy element powinien być wcześniej niż drugi
- 0, jeżeli elementy są równe
- 1, jeżeli pierwszy element powinien być później niż drugi

Przy dodawaniu nowego elementu do słownika, zostanie porównany po kolei z każdym elementem i dodany przed pierwszym elementem, z którym porównanie da wynik -1, lub na koniec słownika.

3.11.2 Tworzenie nowego słownika

Słownik można stworzyć na 2 sposoby:

- poprzez literał - tworzymy słownik poprzez nawias klamrowy i wpisując jego zawartość. Słownik wtedy przyjmuje funkcję *defaultSort* jako funkcję sortującą (zwracającą zawsze 1). Wtedy elementy są posortowane według kolejności dodawania do słownika:

```
{
    literal_dict = {"first": "Hello"}, ("second": 10)};

    literal_dict.add("another", 5);
    // element dodany na koniec słownika
}
```

- poprzez konstruktor - tworzymy nowy słownik, wołając jego konstruktor. Jako argument podajemy własną funkcję sortującą:

```
{
    /* funkcja sortująca w odwrotnej kolejności
       alfabetycznej kluczy */
    my_sort = function(item1, item2) {
        if (item1.key() < item2.key()) {
            return 1;
        }
        if (item1.key() > item1.key()) {
            return -1;
        }
        return 0;
    }

    my_dict = Dict(my_sort);

    my_dict.add("key", 5);
}
```

```

        my_dict.add("zebra", "hello");
        // element dodany na początek słownika, zgodnie z regułą
    }

```

3.11.3 Metody

Słownik zawierać będzie następujące funkcje wbudowane:

- *addNew(key, value)* - utworzenie nowego obiektu *Item* i dodanie go do słownika
- *add(item)* - dodanie istniejącego obiektu *Item* do słownika
- *remove(key)* - usunięcie ze słownika elementu o podanym kluczu
- *contains(key)* - sprawdzenie, czy słownik zawiera element o podanym kluczu
- *get(key)* - uzyskanie obiektu o podanym kluczu
- *length()* - zwraca długość słownika

3.12 Zapytania LINQ

Na słowniku i liście można wykonywać również zapytania LINQ o konstrukcji:

```

from <identyfikator> in <nazwa_słownika>
select <wartości>
where <warunek>
order by <element> <ewentualnie descending>

```

Zapytania zwracają **listę** wybranych elementów. Jeżeli dla jednego obiektu z kolekcji pobieramy kilka wartości, otrzymujemy listę list z pobranymi wartościami. Przykładowe zapytania:

```

{
    my_dict = {
        ("Warszawa", 2000000),
        ("Tokio", 37000000),
        ("Delhi", 30000000),
        ("Szczecinek", 40000),
        ("Buenos Aires", 15000000)
    }

    small_cities = from city in my_dict
        select city.key(), city.value() / 1000
        where city.value() < 5000000
        order by city.key() descending;

    // small_cities = [{"Warszawa", 2000}, {"Szczecinek", 40}]
}

```

4 Obsługa błędów

Na każdym etapie interpretowania kodu może występować błąd, przekazywany do modułu obsługi błędów. Błędy będą wyglądać następująco:

- błąd braku zgodności typów:

```
{
    a = 10;
    b = "Hello" + a;
}
```

ERROR - line 3, col 17 - Can't perform '+' on types 'String' and 'Int'

- nieznanym identyfikator:

```
{
    a = 10;
    c = 10 + b;
}
```

ERROR - line 3, col 14 - 'b' is not recognized in this scope

- błąd syntaktyczny:

```
{
    a = 10 b = a + 10;
}
```

ERROR - line 2, col 10 - ';' missing

- błędna ilość argumentów funkcji:

```
{
    my_func = function(arg1, arg2) {
        return arg1 + arg2 / 2;
    }

    a = my_func(10, 100, 1);
    b = my_func(1);
}
```

ERROR - line 6, col 24 - 'my_func' takes 2 arguments, 3 given

ERROR - line 7, col 18 - 'my_func' takes 2 arguments, 1 given

- błąd rekursji:

```
{
    recursive = function(a) {
```

```

        return recursive(a+1);
    }

    recursive(1);
}

```

ERROR - line 3, col 16 - reached maximum recursion depth

5 Sposób uruchomienia

Programy można uruchamiać na 2 sposoby:

- z pliku .psc:

```

$ psc program.psc
> Hello from file!

```

Plik program.psc:

```

print("Hello World");

```

- ze strumienia tekstowego:

```

$ echo "a = 10; print(a + 10);" | psc
> 20

```

Interpreter będzie miał specjalną flagę `-c`, po której należy podać plik konfiguracyjny `.json`. Umożliwia to modyfikowanie niektórych parametrów interpretera, np. dozwoloną głębokość rekursji, maksymalne długości identyfikatorów itp.:

```

$ psc program.psc -c config.json
> Hello from file with configured interpreter!

```

6 Składnia realizowanego języka

6.1 Słowa kluczowe języka

- and
- or
- if
- else
- True
- False

- function
- return
- while
- for
- from
- in
- select
- where
- order
- by
- descending

6.2 Gramatyka

Opis gramatyki umieszczony został w repozytorium jako */intro_project/grammar.txt*.

7 Testowanie

Każdy moduł interpretera zostanie zbadany testami jednostkowymi przy użyciu biblioteki *pytest*. Sprawdzone zostanie nie tylko poprawność działania, ale i odporność na błędy. Przykładowy program wykorzystujący podstawowe funkcje języka:

```
// Funkcja porównująca miasta alfabetycznie rosnąco
alphabetical_sort = function(item1, item2) {
    if (item1.key() < item2.key()) {
        return -1;
    }
    if (item1.key() > item2.key()) {
        return 1;
    }
    return 0;
};

// Funkcja sprawdzająca, czy populacja miasta przekracza 5 milionów
is_large = function(value) {
    return value > 5000000;
};

// Tworzenie słownika miast z populacją
cities = Dict(alphabetical_sort);
cities.addNew("Warszawa", 2000000);
cities.addNew("Tokio", 37000000);
```

```

cities.addNew("Delhi", 30000000);
cities.addNew("Szczecinek", 40000);
cities.addNew("Buenos Aires", 15000000);

// Wydruk wszystkich miast i ich populacji
print("Miasta i populacja:");
for city in cities {
    print("- ", city.key(), ":", city.value());
}

// Filtrowanie małych miast do nowej listy za pomocą zapytania LINQ
small_cities = from city in cities
    select city.key(), city.value() / 1000
    where city.value() < 5000000
    order by city.key() descending;

// Wydruk przefiltrowanej listy
print("Małe miasta (< 5 mln:");
for entry in small_cities {
    print("- ", entry.get(0), " (" , entry.get(1), " tys.)");
}

// Dodanie miasta jeśli go nie ma
if (!cities.contains("Berlin")) {
    cities.addNew("Berlin", 3800000);
}

// Sprawdzenie typu danych
print("Typ cities:", typeof(cities));
print("Typ small_cities:", typeof(small_cities));

// Kopiowanie listy
backup_list = small_cities.copy();
print("Backup:", backup_list);

```

Przykładowy program z wykorzystaniem różnych typów danych w słowniku:

```

// Funkcja porównująca elementy na podstawie typu ich wartości (alfabetyczni
type_sort = function(item1, item2) {
    type1 = typeof(item1.value());
    type2 = typeof(item2.value());

    if (type1 < type2) {
        return -1;
    }
    if (type1 > type2) {
        return 1;
    }
    return 0;
};

```

```

// Tworzenie słownika z różnorodnymi kluczami i wartościami
mixed_dict = Dict(type_sort);

mixed_dict.addNew("alpha", 42); // Int
mixed_dict.addNew(100, [1, 2, 3]); // List
mixed_dict.addNew("pi", 3.14); // Float
mixed_dict.addNew("info", "tekst"); // String
mixed_dict.addNew(5, {"key": "value"}); // Dict

// Iteracja i wypisywanie posortowanych elementów słownika
print("Elementy posortowane wg typu wartości:");
for element in mixed_dict {
    key = element.key();
    val = element.value();
    print("- key:", key, "| type:", typeof(val), "| value:", val);
}

```